SIMATS ENGINEERING

Department of Computer Science and Engineering ASSESSMENT

TOOL 1 – EXPERIMENTS

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO3 – Precision (BL3)	**Assessment:**	Experiments
Weightage:	45%	**Total Marks:**	100
CO3 Statement:	Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT).		
Student Name:	Hardhik Shankar	**Reg. No.:**	192312417
Section / Batch:	2023	**Date:**	11-08-2026

Instructions to Students

- Identify the relevant concepts of register transfers, ALU operations, bus organization, pipelining, hazards, and superscalar processing.
- Analyze the execution of data transfers, arithmetic and logic operations, instruction processing, and parallel execution.
- Apply suitable techniques such as multiple buses, pipelining, stalling, forwarding, branch prediction, and speculative execution.
- Compute performance measures such as execution time, clock cycles, speedup, throughput, and resource utilization.
- Interpret the results by evaluating performance improvements, bottlenecks, and the suitability of each architectural technique.

QUESTIONS

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyse how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyse a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Code of Conduct

I R Hardhik Shankar (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Q. No	Understanding of Concepts(25)	Experimental Design (30)	Use of Tools(20)	Analysis of Results (15)	Documentation (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	

5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	**/ 30**	**/ 20**	**/ 15**	**/ 10**	**/ 100**

Course Coordinator **Faculty Incharge**

1. Register Transfer Operations and ALU

The designed system consists of multiple registers such as R1, R2, R3 and R4, an Arithmetic Logic Unit (ALU), a common data path and a control unit. The registers are used to temporarily store data during processing, while the ALU performs arithmetic and logical operations. Initially, R1 may contain the value 25 and R2 may contain the value 10. The data can be transferred between registers using operations such as **R3 ← R1** and **R4 ← R2**. After transferring the operands, the ALU performs operations such as addition, subtraction, AND and OR. For example, the operation **R3 ← R1 + R2** causes the contents of R1 and R2 to be supplied to the ALU, where they are added, and the result 35 is stored in R3. Similarly, subtraction can be performed using **R4 ← R1 − R2**, while logical operations can be performed using **R3 ← R1 AND R2** and **R4 ← R1 OR R2**. The control unit generates the control signals required to select the source registers, ALU operation and destination register.

The sequence of register transfers demonstrates how data moves through the processor datapath during instruction execution. The ALU acts as the main processing unit because it receives operands from registers, performs the required operation and produces the result. Registers provide fast temporary storage, reducing the need to access main memory for every operation. During simulation, the contents of each register can be monitored after every clock cycle to verify that the correct transfer and operation have occurred. This system demonstrates the basic working principle of a processor datapath, where **registers store data, the ALU processes data, and the control unit coordinates the entire operation**.

2. Single-Bus and Multi-Bus Organization

A computer system can be organized using either a single-bus or multi-bus architecture for transferring data between registers, memory, I/O devices and the ALU. In a **single-bus organization**, all components share one common communication path. For example, if data has to be transferred from R1 to R2, the contents of R1 are first placed on the common bus and then loaded into R2. If an arithmetic operation is required, the operands may have to be transferred through the same bus in separate steps. This means that only a limited number of transfers can occur at a time. Memory and I/O operations also use the same bus, so simultaneous requests can result in delays. The single-bus architecture is simple, inexpensive and easy to control, but its main disadvantage is the communication bottleneck created by the shared bus.

In a **multi-bus organization**, two or more independent buses are provided for data transfer. This allows multiple operations to take place simultaneously. For example, two operands can be transferred from R1 and R2 to the ALU through separate buses, while the ALU result can be transferred to another register through a third bus. Memory and I/O operations can also be performed without blocking every internal processor transfer. As a result, multi-bus systems provide greater parallelism and reduce the number of clock cycles required for data movement. Although multi-bus organization requires additional hardware, multiplexers and more complex control logic, its higher data-transfer capability makes it more efficient for high-performance processors. Therefore, **single-bus systems are simpler and cheaper, whereas multi-bus systems provide better performance by enabling parallel data transfers and reducing bottlenecks**.

3. Five-Stage Instruction Pipeline

   A five-stage instruction pipeline divides the execution of an instruction into five smaller stages: **Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB)**. During the Fetch stage, the processor obtains the instruction from memory. During Decode, the instruction is interpreted and the required operands are identified. The Execute stage performs the required arithmetic, logical or address calculation operation. The Memory stage is used for accessing memory when required, and finally the Write-back stage stores the result in the destination register. The main advantage of pipelining is that these stages can operate on different instructions at the same time.

   For example, if four instructions are executed using a five-stage pipeline, the first instruction requires five cycles to pass through all stages. However, once the pipeline is filled, a new instruction can complete approximately every clock cycle. The total number of cycles for four instructions is **5 + 4 − 1 = 8 cycles**. In a non-pipelined processor where each instruction requires five cycles independently, the total would be **4 × 5 = 20 cycles**. Therefore, the speedup is **20/8 = 2.5 times** for this example. As the number of instructions increases, the performance approaches the ideal pipeline speedup. Pipelining therefore improves processor throughput by allowing different instructions to occupy different stages simultaneously. However, hazards, branch instructions and resource conflicts can introduce stalls and reduce the actual performance compared with the ideal case.

4. Pipeline Hazards and Hazard Resolution

   Pipeline hazards are conditions that prevent instructions from executing smoothly in an overlapped pipeline. The three major types are **data hazards, control hazards and structural hazards**. A data hazard occurs when an instruction depends on the result produced by an earlier instruction. For example, if I1 performs `ADD R1, R2, R3` and I2 immediately uses R1, I2 may need to wait until I1 produces the required result. This can be handled using **data forwarding**, where the result is directly forwarded from an earlier pipeline stage to the instruction that requires it, avoiding unnecessary waiting. If forwarding is not possible, the processor can insert a stall or bubble into the pipeline.

   A control hazard occurs when the processor encounters a branch or jump instruction because it does not immediately know which instruction should be fetched next. **Branch prediction** can be used to predict whether the branch will be taken or not taken. If the prediction is correct, the pipeline continues without a significant delay. If the prediction is incorrect, the incorrectly fetched instructions are removed from the pipeline and the correct instructions are fetched. A structural hazard occurs when two instructions require the same hardware resource at the same time. For example, if two instructions require access to a single memory unit simultaneously, one instruction must wait. This can be solved by adding additional hardware resources or using pipeline stalls. Therefore, forwarding, branch prediction, resource duplication and controlled stalling help reduce the effect of hazards and improve overall pipeline performance.

5. Superscalar, Out-of-Order and SMT Execution

   A **superscalar processor** is capable of fetching, decoding and executing multiple instructions during a single clock cycle by using multiple execution units. For example, a four-issue superscalar processor may theoretically execute up to four independent instructions in one cycle. This increases instruction-level parallelism and provides significantly higher throughput than a scalar processor. However, not all instructions are independent, so the processor must identify dependencies and determine which instructions can safely execute in parallel. Multiple functional

units such as ALUs, floating-point units and load/store units allow different types of instructions to execute simultaneously.

Out-of-order execution further improves performance by allowing independent instructions to execute before earlier instructions that are waiting for data or resources. For example, if instruction I1 is waiting for a memory value but instruction I2 is independent, I2 can execute first instead of leaving the execution units idle. Once I1 receives its required data, it can continue execution. **Speculative execution** works with branch prediction by allowing the processor to execute instructions from a predicted path before the branch outcome is completely known. If the prediction is correct, valuable time is saved. If it is incorrect, the speculative results are discarded and execution continues from the correct path.

Hyper-threading and Simultaneous Multithreading (SMT) extend this concept by allowing multiple software threads to share the resources of a physical processor core. When one thread is waiting for memory or another resource, another thread can use available execution resources. This improves utilization of the ALUs, registers, caches and other processor resources. Therefore, superscalar execution provides instruction-level parallelism, out-of-order execution improves the scheduling of independent instructions, speculative execution reduces branch delays, and SMT provides thread-level parallelism. When combined, these techniques significantly increase processor throughput and make better use of available hardware resources.

6.